Capítulo 1: Inicio y control de versiones

Introducción

1.1 ¿Qué es el control de versiones y por qué se necesita?

Un sistema de control de versiones registra los cambios hechos sobre un conjunto de archivos a lo largo del tiempo, de modo que se puedan recuperar versiones anteriores. Sin él, es común terminar con copias como informe_final_v2_definitivo.docx, lo que genera confusión sobre cuál es la versión vigente.

Sus aportes principales son:

- **Historial:** cada cambio queda registrado con su fecha, su autor y una descripción.
- **Trazabilidad:** se puede saber quién modificó qué y por qué, lo que facilita revisar y auditar el trabajo.
- **Reversión de errores:** si un cambio daña el proyecto, se puede volver a un estado anterior que funcionaba.
- **Colaboración:** varias personas pueden trabajar sobre el mismo proyecto sin sobrescribirse el trabajo.

1.2 Tipos de sistemas de control de versiones

Tipo	Cómo funciona	Limitación o ventaja
Local	Guarda las versiones en una base de datos dentro del mismo computador	Simple, pero no permite colaborar y todo depende de un único equipo
Centralizado	Un servidor central guarda el historial y los usuarios descargan los archivos desde allí	Facilita la colaboración, pero si el servidor falla se pierde el acceso al historial (punto único de fallo)
Distribuido	Cada usuario clona el repositorio completo, con todo su historial	Permite trabajar sin conexión y cualquier copia puede restaurar al servidor

1.3 ¿Qué es Git?

Git es un sistema de control de versiones distribuido y de código abierto, creado por Linus Torvalds. Git pertenece al grupo de los sistemas distribuidos.

Sus características más relevantes son:

- **Instantáneas (*snapshots*), no solo diferencias.** Muchos sistemas guardan la lista de cambios línea por línea. Git, en cambio, toma una "fotografía" del estado completo del proyecto en cada confirmación y, si un archivo no cambió, simplemente enlaza a la versión anterior.
- **Trabajo sin conexión.** Como el historial completo está en el computador del usuario, casi todas las operaciones (consultar el historial, comparar versiones, confirmar cambios) son locales y rápidas. Solo se necesita red para sincronizar con un repositorio remoto.
- **Integridad de los datos.** Cada objeto se identifica con una suma de verificación (*hash*), por lo que Git detecta cualquier alteración o corrupción del contenido.
- **Los cambios suelen ser reversibles**.

1.4 Los tres estados

Un archivo rastreado por Git puede estar en tres estados, y cada uno se relaciona con una zona del proyecto:

Estado	Zona	Significado
Modificado (*modified*)	Directorio de trabajo	El archivo cambió, pero esos cambios aún no se han marcado
Preparado (*staged*)	Área de preparación	Se marcó el archivo para incluirlo en la próxima confirmación
Confirmado (*committed*)	Repositorio (directorio .git)	Los cambios quedaron guardados de forma segura en el historial

El flujo habitual es: modificar archivos, prepararlos con git add y confirmarlos con git commit.

Conclusión

El control de versiones evita la pérdida de trabajo y organiza la colaboración. Git, al ser distribuido y basado en instantáneas, ofrece velocidad, seguridad y autonomía, y su flujo de tres estados es la base de todo lo que se verá en los siguientes capítulos.

Referencia: Chacon, S., & Straub, B. (2014). *Pro Git* (2nd ed.). Apress. https://git-scm.com/book/es/v2